

We will solve the problem in complexity $\mathcal{O}(n \log^2 n)$.

Let $f(i, j) = \min(a_i, \dots, a_j)$ and $g(i, j) = \min(b_i, \dots, b_j)$.

We want to find the interval $[i, j]$ which maximises $f(i, j) \cdot g(i, j) \cdot (j - i + 1)$. Solution is based on *divide-and-conquer* approach: function $solve(lo, hi)$ will find the best interval contained in $[lo, hi]$ that

includes the point mid . It will recursively call $solve(lo, mid - 1)$ and $solve(mid + 1, hi)$.

Let $[l, r]$ be an interval such that $l \in [lo, mid]$ and $r \in [mid, hi]$. We consider four cases:

1. $f(l, r)$ and $g(l, r)$ are determined by the left half, i.e. $f(l, r) = f(l, mid)$ and $g(l, r) = g(l, mid)$.
2. $f(l, r)$ and $g(l, r)$ are determined by the right half, i.e. $f(l, r) = f(mid, r)$ and $g(l, r) = g(mid, r)$.
3. $f(l, r)$ is determined by the left, and $g(l, r)$ by the right half.
4. $f(l, r)$ is determined by the right, and $g(l, r)$ by the left half.

First and second cases are analogous, and so are third and fourth, so we will describe the solution for the first and the third case.

Case 1: $f(l, r)$ i $g(l, r)$ are determined by the left half.

Note that this implies $f(mid, r) \geq f(l, mid)$ i $g(mid, r) \geq g(l, mid)$.

This case can easily be solved using the *two pointers* method in complexity $\mathcal{O}(hi - lo)$, i.e. $\mathcal{O}(n \log n)$ in total.

Case 3: $f(l, r)$ is determined by the left, and $g(l, r)$ by the right half.

Note that this implies $f(mid, r) \geq f(l, mid)$ i $g(mid, r) \leq g(l, mid)$.

This case is much more complex, and it can also be solved in $\mathcal{O}(n \log^2 n)$ total complexity. We will first describe a $\mathcal{O}(n \log^3 n)$ solution, and then optimise it.

For some l , let's denote the interval of possible r -s by $[a(l), b(l)]$. Borders $a(l)$ and $b(l)$ can again be found using the two pointers method.

We have

$$\begin{aligned} & f(l, mid) \cdot g(mid, r) \cdot (r - l + 1) = \\ &= f(l, mid) \cdot (-l + 1) \cdot g(mid, r) + f(l, mid) \cdot g(mid, r) \cdot r = \\ &= \langle (f(l, mid) \cdot (-l + 1), f(l, mid)), (g(mid, r), g(mid, r) \cdot r) \rangle, \end{aligned}$$

where $\langle (a, b), (c, d) \rangle$ represents the dot product of vectors (a, b) and (c, d) . Note that the first vector $((l, mid) \cdot (-l + 1), f(l, mid))$ depends only on l , and the second vector $(g(mid, r), g(mid, r) \cdot r)$ depends only on r .

For a fixed l , we want to find the $r \in [a(l), b(l)]$ that maximises the dot product. If r was not limited to that interval, we could calculate the convex hull of the set of second points, and use ternary search to find the optimal r for each l . But now, we can sort the points by (for example) the first coordinate and build a segment tree that has the convex hull of the corresponding points in each node. So, for each l we will look at $\mathcal{O}(\log n)$ nodes in the segment tree, and do a ternary search on the hull in each node in $\mathcal{O}(\log n)$ complexity. This gives us the total complexity of $\mathcal{O}(n \log^3 n)$, because we also need to count in the divide-and-conquer.

Now we want to get rid of the binary search. Notice that points $f(l, mid) \cdot (-l + 1, 1)$ move counterclockwise as l increases, and so the optimal point on the hull will also move counterclockwise! For each node in the segment tree, we can answer its queries in amortised linear complexity, by keeping a pointer to the optimal point for the last query. This gives us total complexity of $\mathcal{O}(n \log^2 n)$,